Authored by: Arpit Raj, LNMIIT Jaipur
Date: October 23, 2026
© 2026 Arpit Raj. All rights reserved.

Advanced OOP: Encapsulation, Structs & Friend Functions

Another benefit of getter/setter
-> adds an additional security layer for controlled access of data

Eg

```
void set/get( ) {
    // authorization layer
    this->property = value
}
```

Data Hiding vs Encapsulation

Data hiding	Encapsulation
restricts direct access to internal data	organizes related data & operations into a unit and defines how that unit is accessed
focus on prevention of unwanted access	focus on design and boundaries of unit
achieved through access controls like `private`	achieved through class design, access controls, controlled interfaces
part of encaps	a design principle

- **Encapsulation** :- How is state organized and access is controlled. -> `private` balance, `public` deposit
- **Abstraction** :- what functionality is exposed and what implement details can be ignored. -> call withdraw without know internal logic
- **Modularity** :- how is program divided into manageable components. -> accounts, employees etc

Access specifiers & Protected Modifier

Protected access modifier can be accessed by a derived class, subject to inheritance rules.

Eg

```
class BankAccount {
protected:
    int bal = 5000;
};

class SavingsAccount : public BankAccount {
public:
    void showbal() {
        cout << Balance; // valid
    }
};
```

Class vs Struct & Inheritance Traps

`class` and `struct` are almost identical in capabilities, the important default difference is :-

- default member and inheritance access is `private` for `class` and `public` for `struct`.

Encapsulation can be done via struct too in same way.

⚠️ **INTERVIEW TRAP**

- the default access depends on parent type

```
class Parent {
public:
    int x = 10;
};

class child1 : Parent {
};

struct child2 : Parent {
};

main() {
    child1 a;
    child2 b; cout<<a.x and cout<<b.x both gives error , because default
behaviour depend on parent
```

to solve error need to write as

```
struct child2 : public Parent {
};
```

Friend Functions & Classes

-> a friend function is an alternative way to access `private` and `protected` members without calling getter and setter methods.

- use getter setters for maintainable and safe external access because they give controlled, standardized interface to read/modify, often allow a validation logic / authorization layer
- friend functions used when special helper routines need high speed or multi class access without exposing to entire public
- friend functions are trusted outsiders, getters and setters belong strictly to a single class and friend function can be declared as friends of multiple classes, thats why they have multi class access

friend function syntax

```
class Bank {
private:
    int bal = 1000
public:
    friend void fun1(Bank B1);
};

now void fun1(Bank B1) {
    cout << B1.balance
}

main() {
    Bank B;
    fun1(B)
}
```

> **Note:** `friend` keywords grant access and not make a function member of a class.

friend class syntax

similarly we can make an entire class a friend of another class, Eg

```
class Bank {
private:
    int bal = 1000;
public:
    friend class Auditor;
};

class Auditor {
public:
    void inspect(Bank Bn) {
        cout << Bn.bal;
    }
};

main() {
    Bank B
    Auditor A
    A.inspect(B)
}
```

Friendship Rules

1. A declares B a friend, B can get A private, A cant automatically get B private info -> **one sided love**
2. A declare B a friend, C is derived from B, C does not get friendship of A -> **love is not inherited**
3. A loves B, B loves C, does not mean A loves C -> **love is not transitive**

Note

```
class Test {
private:
    int x = 10;
public:
    friend void display();
};

void display() {
    cout << x; // -> ERROR
} function has access but it needs an object as a parameter , so write Test t
within bracket and cout<<t.x
```